

Project Overview

PSI4 is an open-source quantum chemistry software package used to perform computationally intensive electronic structure calculations.

As these calculations scale to larger systems and more CPU threads, inefficient code can increase execution time and waste computing resources.

Project Objectives

- Develop an interactive dashboard for PSI4 developers to visualize and analyze performance trends
- Identify code-level bottlenecks and improve parallel performance

Workflow

Regression tests/Run tests → Collect Data → Dashboard → Identify Bottlenecks → Optimize and Validate

Goals and Milestones

Dashboard Data Visualization

- Built a performance data pipeline to collect timing and SCF iteration data
- Developed an interactive dashboard with graphs to visualize performance trends across test cases, commits, and thread counts

Performance Analysis & Optimization

- Ran selected test cases on the ICE cluster and used the dashboard to identify performance and scaling bottlenecks
- Traced bottlenecks to their source and optimized the underlying code to improve performance

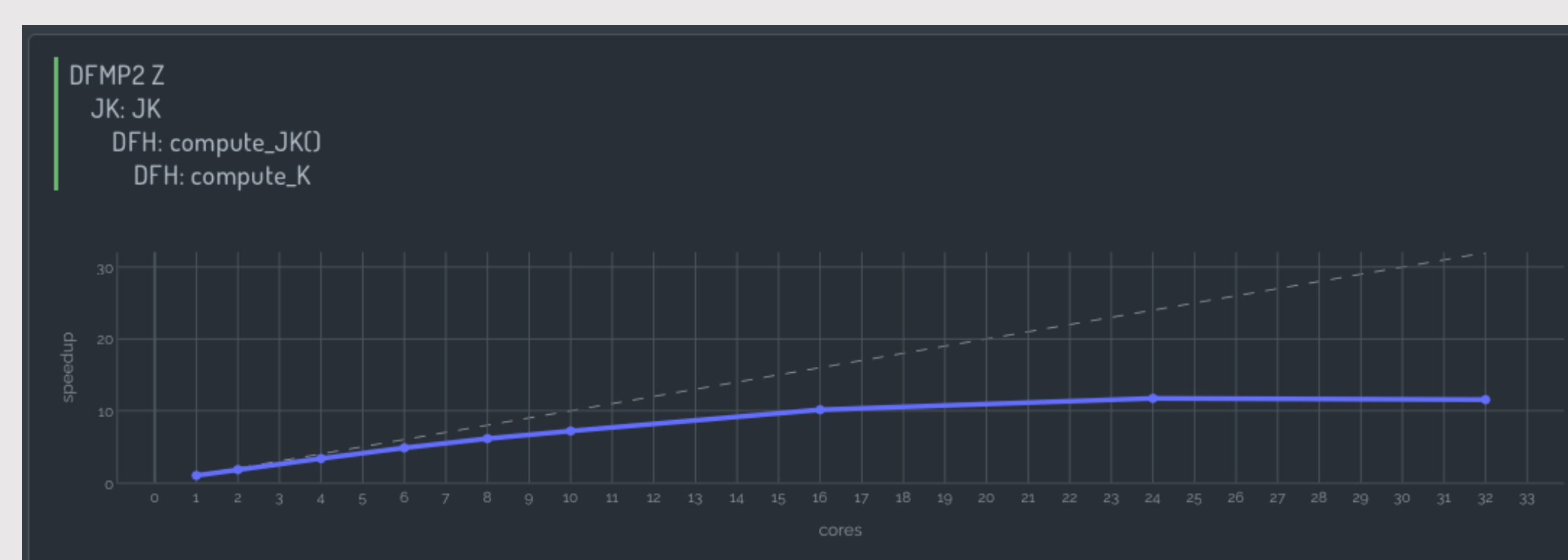
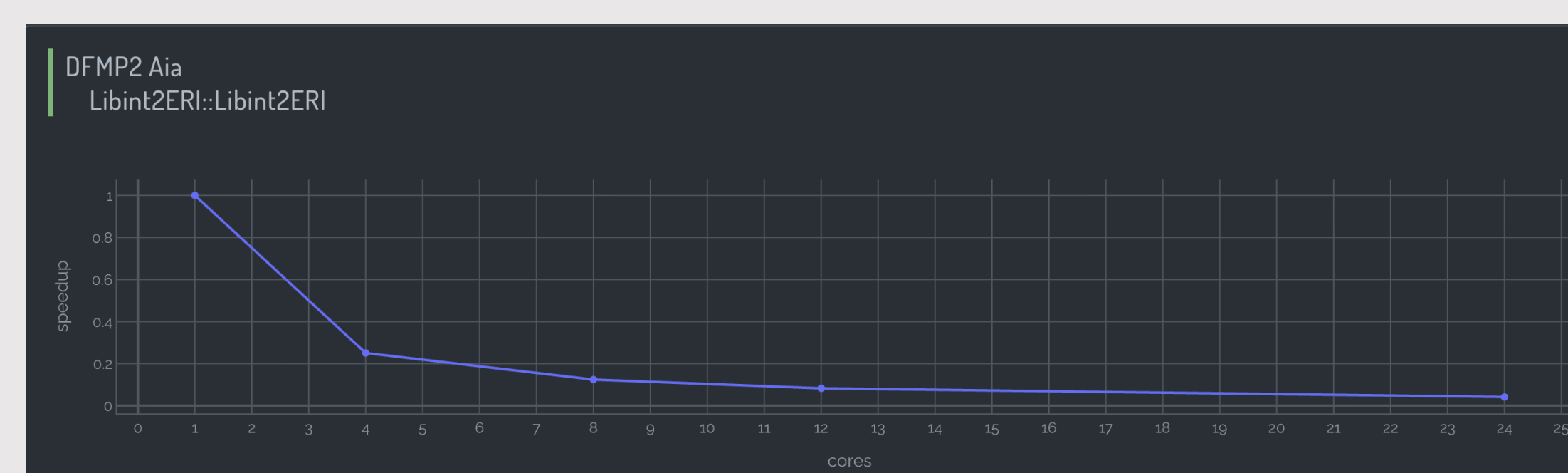
Dashboard

- Collected timing and SCF performance data from regression tests
- Stored data in JSON following the timing hierarchy, preserving parent-child relationships
- Each timer record included metadata such as version number to track commit history
- The data was loaded into a Pandas DataFrame
- It was presented as Plotly graphs on a Plotly Dash app
- Provided multiple pages and filters to view the data as desired

Dashboard Pictures



Speedup Plots



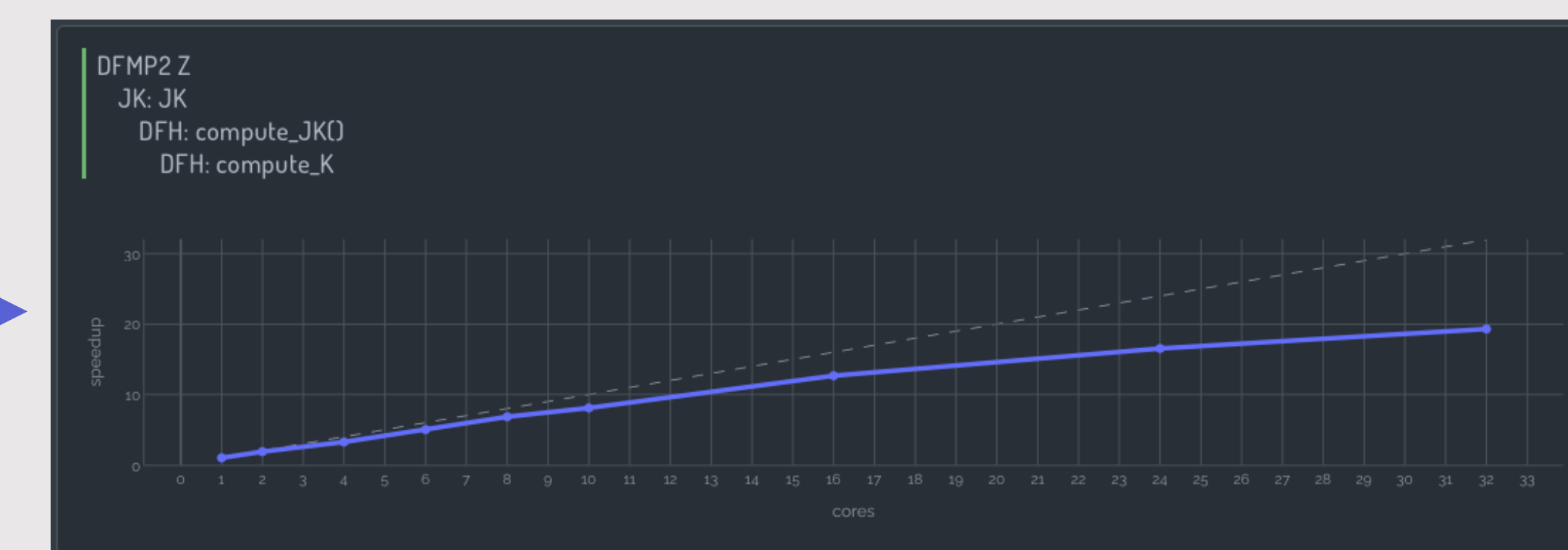
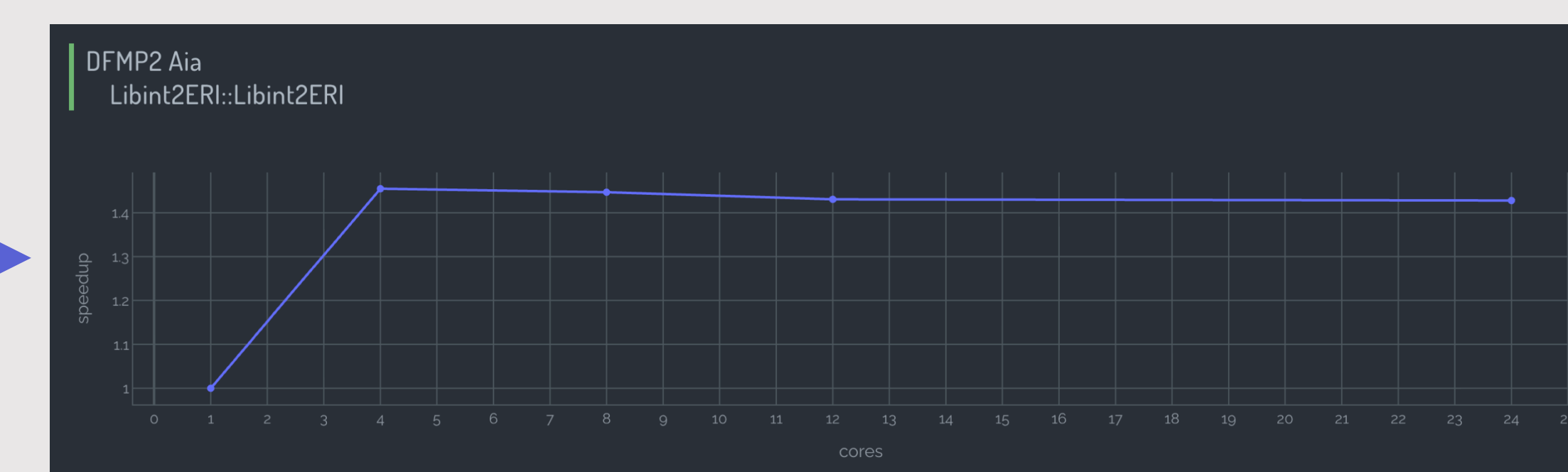
Performance Optimization

Analysis

- Targeted DFMP2 library for performance improvements
- Investigated DFMP2 Aia timer
- Found repeated time-consuming Libint2ERI object construction across threads
- Reused and cloned the constructor objects to reduce redundant construction
- Most of the code was already optimized for parallel processing
- Further improvements came from memory alignment
- NUMA architecture degraded performance since all nodes had to read from the memory of one single node
- `numactl --interleave=all` disperses the process's pages across all nodes

Result

- Improved Libint2ERI timer 1→24-core speedup from 0.04x to 1.43x
- Improved overall 1→32 core speedup from 11.6x to 14.3x
- Improved compute_K() subroutine (30% of single-core total runtime) 1→32 core speedup from 11.6x to 19.3x



Open Source Outcomes

- Explored a dense codebase to get familiar with the project
- Made Pull Requests to push changes
- Iterated over the PR to address concerns raised by maintainers

Future Work

- In-code changes to replicate the effect of the `numactl --interleave=all` command on PACE
- Explore other libraries for more performance optimization opportunities
- Dashboard enhancements